FORMATION « PYTHON EN 24 HEURES » | SÉANCE 10

Mini-projet guidé : un système bancaire à trois classes

Chapitre 19 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Lire un cahier des charges et en déduire les classes nécessaires
- Faire dialoguer des objets entre eux (Client, Compte, Banque)
- Centraliser les validations et préparer les exceptions métier
- Conserver une trace des opérations : l'historique de transactions
- Produire un rapport lisible depuis les données des objets

Rappel des séances 08 et 09

Le projet assemble tout : encapsulation de la séance 08, méthodes spéciales de la séance 09, et deux séances de préparation pour rien d'autre que bâtir.

- `__init__`, self, attributs d'instance et de classe
- `_interne` et `__privé`, `@property` avec setter validant
- `@classmethod` pour les fabriques, `__str__` pour la présentation
- `super()` et l'héritage quand la relation EST UN est vraie
- `__len__` et `__getitem__` pour transformer une classe en conteneur

Le cahier des charges

Le Client

- Un numéro unique et un nom
- Peut détenir plusieurs comptes
- Se présente proprement (__str__)

Le Compte

- Un solde jamais modifiable de l'extérieur
- Dépôt, retrait avec validation
- Un historique de toutes les opérations

La Banque

- Ouvre les comptes et les numérote
- Effectue les virements entre comptes
- Édite le rapport général

Architecture : qui connaît qui ?

```
# La Banque détient les comptes ; le Client est référencé
banque = Banque()
compte = banque.ouvrir_compte("Awa", 1000)
banque.ouvrir_compte("Kofi", 250)

banque.virer(compte.numero, cible_numero, 400)
print(banque.rapport())
```

Règle d'or : chaque classe ne connaît que ses attributs. La Banque orchestre ; le Compte n'appelle jamais la Banque.

Étape 1 : la classe Client

```python
class Client:
    def __init__(self, numero, nom):
        self.numero = numero
        self.nom = nom

    def __str__(self):
        return f"Client {self.numero} : {self.nom}"

awa = Client(1, "Awa")
print(awa)
```

Pas de logique, pas de validation complexe : le Client est une fiche d'identité. La simplicité d'une classe se juge à ce qu'elle refuse de faire.

Étape 2 : le Compte et son historique

```python
class Compte:
    def __init__(self, numero, client, solde=0):
        self.numero = numero
        self.client = client
        self._solde = solde
        self.historique = [f"Ouverture : {solde} F"]

    def deposer(self, montant):
        if montant <= 0:
            raise ValueError("montant invalide")
        self._solde += montant
        self.historique.append(f"Dépôt : +{montant} F")
```

Chaque opération validée laisse une trace : l'historique est le fil d'audit du compte. La séance 11 transformera ces ValueError en exceptions métier.

Étape 3 : la Banque orchestre

```
class Banque:
    def __init__(self):
        self.comptes = {}
        self.prochain_numero = 1
    def ouvrir_compte(self, nom, solde=0):
        client = Client(self.prochain_numero, nom)
        compte = Compte(self.prochain_numero, client, solde)
        self.comptes[compte.numero] = compte
        self.prochain_numero += 1
        return compte
    def virer(self, source, cible, montant):
        depart = self.comptes[source]
        arrivee = self.comptes[cible]
        depart.retirer(montant)
        arrivee.deposer(montant)
```

virer appelle retirer PUIS deposer : si retirer lève une exception, rien n'a été déposé. L'ordre des opérations est une décision de sécurité.

Le rapport : objets en texte

- Le rapport parcourt tous les comptes
- Chaque compte fournit sa ligne via ___str___ ou une méthode resume()
- La Banque ajoute les totaux globaux
- Les f-strings alignent les colonnes : rappel de la séance 03
- Un bon rapport se lit sans avoir ouvert le code

```
def rapport(self):
    lignes = ["Rapport de la banque"]
    total = 0
    for compte in self.comptes.values():
        lignes.append(str(compte))
        total += compte.solde
    lignes.append(f"Total : {total} F")
    return "\n".join(lignes)
```

Étape 4 : virement par numéros

```
class Banque:
    def virer_par_numeros(self, source, cible, montant):
        depart = self.compte(source)      # KeyError si inconnu
        arrivee = self.compte(cible)
        return virer(depart, arrivee, montant)

b = Banque()
c1 = b.ouvrir_compte("Awa", 1000)
c2 = b.ouvrir_compte("Kofi", 250)
b.virer_par_numeros(1, 2, 400)
print(c1.solde, c2.solde)   # 600 650
```

La Banque traduit les numéros en objets puis réutilise le virement déjà validé : chaque couche ne fait qu'un travail, le sien.

Étendre le projet : les pistes

- **Intérêts** : une méthode ‘appliquer_interets(taux)‘ sur les comptes créditeurs
- **Découvert autorisé** : un plafond par compte, vérifié au retrait
- **Exceptions métier** : CompteInconnuError, SoldeInsuffisantError (séance 11)
- **Persistance** : sauvegarder les comptes dans un fichier (séance 07)
- **Tests** : chaque règle du cahier des charges devient un assert

Astuce

Avancer par tests, pas par espoir

Après CHAQUE étape du TP, exécutez les cellules de vérification de toutes les étapes précédentes : si une ancienne étape casse, vous le savez immédiatement, pas à la fin. C'est le réflexe des tests automatisés que le Tome 2 systématisera avec unittest.

Attention

Ne contournez pas les méthodes

Dans le rapport, écrire ‘compte._solde‘ fonctionne mais casse l’encapsulation : si demain le solde devient calculé (solde + découvert), le rapport mentira. Utilisez toujours l’interface publique : ‘compte.solde‘ via @property, et les méthodes validées. La discipline d’aujourd’hui évite le bug de demain.

À retenir

L'essentiel de la séance 10

Un mini projet se construit par étapes : une classe fiche (Client), une classe métier validée (Compte), un orchestrateur (Banque). Les classes se parlent par méthodes publiques uniquement, chaque opération laisse une trace, et le rapport assemble les présentations individuelles. Ce squelette à trois classes s'adapte à la bibliothèque, au stock ou au carnet de notes.

plète

Le TP EST le projet : vous bâtirez la banque entière en cinq étapes, dans l'ordre. Chaque étape repose sur la précédente ; les tests de chaque exercice valident l'étape en cours. Les numéros de compte vont de 1 à 3 dans les tests.

- **Exercice 1 (Basique)** : Étape 1 : la classe Client
- **Exercice 2 (Intermédiaire)** : Étape 2 : la classe Compte avec historique
- **Exercice 3 (Intermédiaire)** : Étape 3 : les virements
- **Exercice 4 (Intermédiaire)** : Étape 4 : la classe Banque
- **Exercice 5 (Défi)** : Étape 5 : intérêts et rapport mensuel

Ouvrez le notebook « Seance_10_mini_projet_bancaire.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 11 : les exceptions. Vos ValueError et KeyError deviennent des exceptions métier nommées, attrapables et documentées. Relisez le traceback de toute erreur levée dans ce TP : savoir le lire est la moitié du chemin.